

SECTION 6

ON-CHIP CACHE MEMORIES

The MC68030 microprocessor includes a 256-byte on-chip instruction cache and a 256-byte on-chip data cache that are accessed by logical (virtual) addresses. These caches improve performance by reducing external bus activity and increasing instruction throughput.

Reduced external bus activity increases overall performance by increasing the availability of the bus for use by external devices (in systems with more than one bus master, such as a processor and a DMA device) without degrading the performance of the MC68030. An increase in instruction throughput results when instruction words and data required by a program are available in the on-chip caches and the time required to access them on the external bus is eliminated. Additionally, instruction throughput increases when instruction words and data can be accessed simultaneously.

As shown in Figure 6-1, the instruction cache and the data cache are connected to separate on-chip address and data buses. The address buses are combined to provide the logical address to the memory management unit (MMU). The MC68030 initiates an access to the appropriate cache for the requested instruction or data operand at the same time that it initiates an access for the translation of the logical address in the address translation cache of the MMU. When a hit occurs in the instruction or data cache and the MMU validates the access on a write, the information is transferred from the cache (on a read) or to the cache and the bus controller (on a write). When a hit does not occur, the MMU translation of the address is used for an external bus cycle to obtain the instruction or operand. Regardless of whether or not the required operand is located in one of the on-chip caches, the address translation cache of the MMU performs logical-to-physical address translation in parallel with the cache lookup in case an external cycle is required.

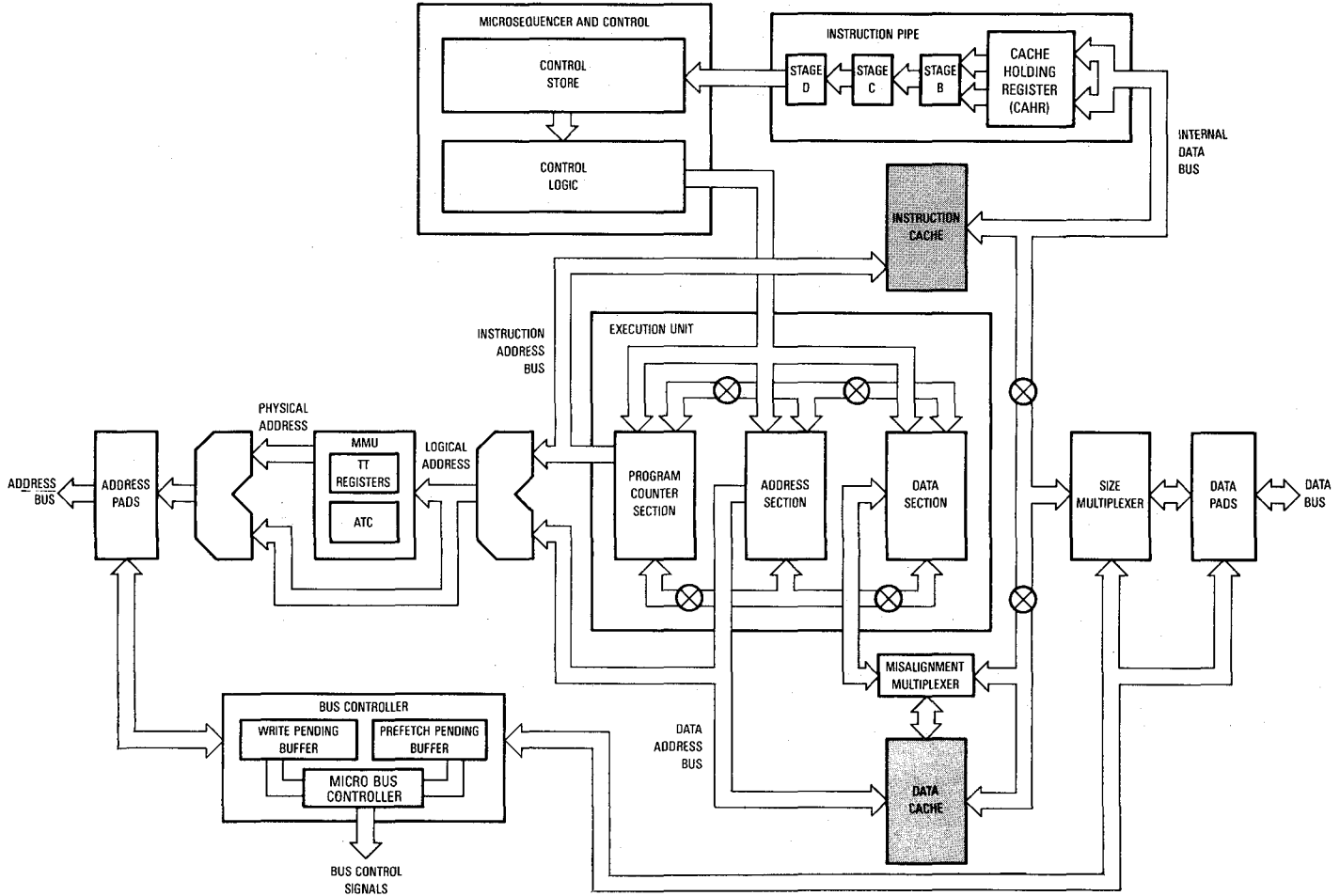


Figure 6-1. Internal Caches and the MC68030

6.1 ON-CHIP CACHE ORGANIZATION AND OPERATION

Both on-chip caches are 256-byte direct-mapped caches, each organized as 16 lines. Each line consists of four entries, and each entry contains four bytes. The tag field for each line contains a valid bit for each entry in the line; each entry is independently replaceable. When appropriate, the bus controller requests a burst mode operation to replace an entire cache line. The cache control register (CACR) is accessible by supervisor programs to control the operation of both caches.

System hardware can assert the cache disable ($\overline{\text{CDIS}}$) signal to disable both caches. The assertion of $\overline{\text{CDIS}}$ disables the caches, regardless of the state of the enable bits in CACR. $\overline{\text{CDIS}}$ is primarily intended for use by in-circuit emulators.

Another input signal, cache inhibit in ($\overline{\text{CIIN}}$), inhibits caching of data reads or instruction prefetches on a bus-cycle by bus-cycle basis. Examples of data that should not be cached are data for I/O devices and data from memory devices that cannot supply a full port width of data, regardless of the size of the required operand.

Subsequent paragraphs describe how $\overline{\text{CIIN}}$ is used during the filling of the caches.

An output signal, cache inhibit out ($\overline{\text{CIOUT}}$), reflects the state of the cache inhibit (CI) bit from the MMU of either the address translation cache entry that corresponds to a specified logical address or the transparent translation register that corresponds to that address. Whenever the appropriate CI bit is set for either a read or a write access and an external bus cycle is required, $\overline{\text{CIOUT}}$ is asserted and the instruction and data caches are ignored for the access. This signal can also be used by external hardware to inhibit caching in external caches.

Whenever a read access occurs and the required instruction word or data operand is resident in the appropriate on-chip cache (no external bus cycle is required), the MMU is completely ignored, unless an invalid translation resides in the MMU at that time (see next two paragraphs). Therefore, the state of the corresponding CI bits in the MMU are also ignored. The MMU is used to validate all accesses that require external bus cycles; an address translation must be available and valid, protections are checked, and the $\overline{\text{CIOUT}}$ signal is asserted appropriately.

An external access is defined as “cachable” for either the instruction or data cache when all the following conditions apply:

- The cache is enabled with the appropriate bit in the CACR set.
- The $\overline{\text{CDIS}}$ signal is negated.
- The $\overline{\text{CIIN}}$ signal is negated for the access.
- The $\overline{\text{CIOUT}}$ signal is negated for the access.
- The MMU validates the access.

Because both the data and instruction caches are referenced by logical addresses, they should be flushed during a task switch or at any time the logical-to-physical address mapping changes, including when the MMU is first enabled. In addition, if a page descriptor is currently marked as valid and is later changed to the invalid type (due to a context switch or a page replacement operation) *entries in the on-chip instruction or data cache corresponding to the physical page must be first cleared (invalidated)*. Otherwise, if on-chip cache entries are valid for pages with descriptors in memory marked invalid, processor operation is unpredictable.

Data read and write accesses to the same address should also have consistent cachability status to ensure that the data in the cache remains consistent with external memory. For example, if $\overline{\text{CIOUT}}$ is negated for read accesses within a page and the MMU configuration is changed so that $\overline{\text{CIOUT}}$ is subsequently asserted for write accesses within the same page, those write accesses do not update data in the cache, and stale data may result. Similarly, when the MMU maps multiple logical addresses to the same physical address, all accesses to those logical addresses should have the same cachability status.

6.1.1 Instruction Cache

The instruction cache is organized with a line size of four long words, as shown in Figure 6-2. Each of these long words is considered a separate cache entry as each has a separate valid bit. All four entries in a line have the same tag address. Burst filling all four long words can be advantageous when the time spent in filling the line is not long relative to the equivalent bus-cycle time for four nonburst long-word accesses, because of the probability that the contents of memory adjacent to or close to a referenced operand or instruction is also required by subsequent accesses. Dynamic RAMs supporting fast access modes (page, nibble, or static column) are easily employed to support the MC68030 burst mode.

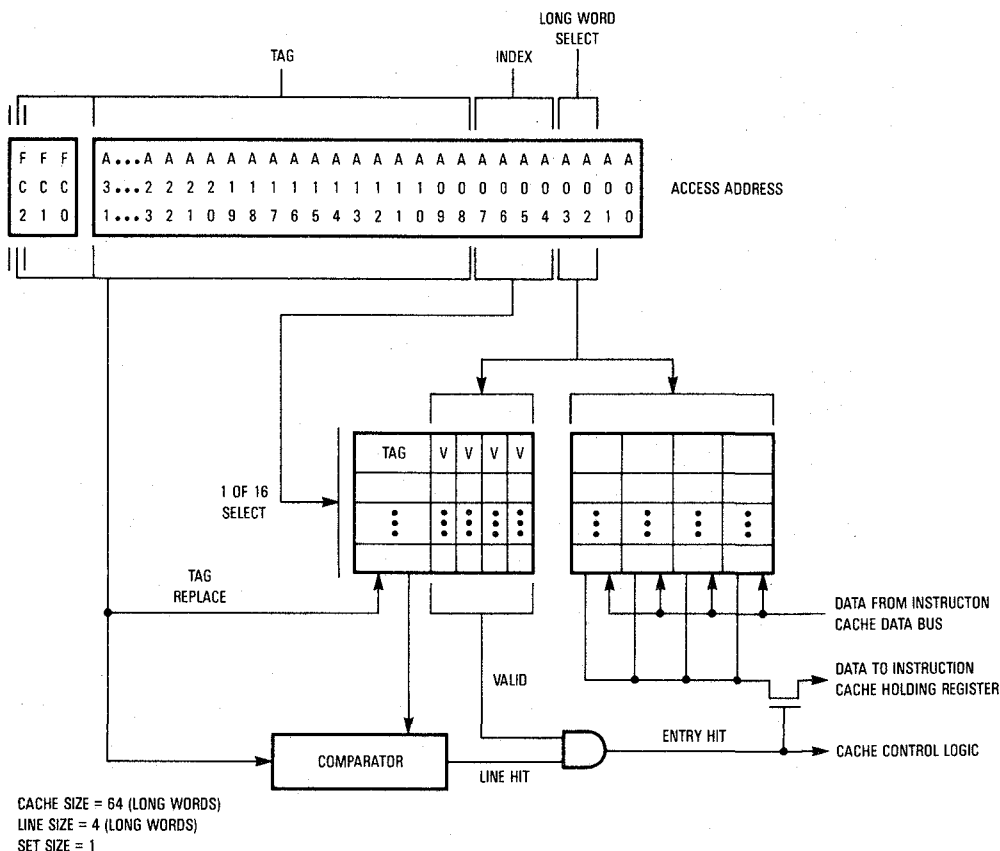


Figure 6-2. On-Chip Instruction Cache Organization

When enabled, the instruction cache is used to store instruction prefetches (instruction words and extension words) as they are requested by the CPU. Instruction prefetches are normally requested from sequential memory addresses except when a change of program flow occurs (e.g., a branch taken) or when an instruction is executed that can modify the status register, in which cases the instruction pipe is automatically flushed and refilled. The output signal **REFILL** indicates this condition. For more information on the operation of this signal, refer to **SECTION 12 APPLICATIONS INFORMATION**.

In the instruction cache, each of the 16 lines has a tag consisting of the 24 most significant logical address bits, the FC2 function code bit (used to distinguish between user and supervisor accesses), and the four valid bits (one

corresponding to each long word). Refer to Figure 6-2 for the instruction cache organization. Address bits A7–A4 select one of 16 lines and its associated tag. The comparator compares the address and function code bits in the selected tag with address bits A31–A8 and FC2 from the internal prefetch request to determine if the requested word is in the cache. A cache hit occurs when there is a tag match and the corresponding valid bit (selected by A3–A2) is set. On a cache hit, the word selected by address bit A1 is supplied to the instruction pipe.

When the address and function code bits do not match or the requested entry is not valid, a miss occurs. The bus controller initiates a long-word prefetch operation for the required instruction word and loads the cache entry, provided the entry is cachable. A burst mode operation may be requested to fill an entire cache line. If the function code and address bits match and the corresponding long word is not valid (but one or more of the other three valid bits for that line are set) a single entry fill operation replaces the required long word only, using a normal prefetch bus cycle or cycles (no burst).

6.1.2 Data Cache

The data cache stores data references to any address space except CPU space (FC = \$7), including those references made with PC relative addressing modes and accesses made with the MOVES instruction. Operation of the data cache is similar to that of the instruction cache, except for the address comparison and cache filling operations. The tag of each line in the data cache contains function code bits FC0, FC1, and FC2 in addition to address bits A31–A8. The cache control circuitry selects the tag using bits A7–A4 and compares it to the corresponding bits of the access address to determine if a tag match has occurred. Address bits A3–A2 select the valid bit for the appropriate long word in the cache to determine if an entry hit has occurred. Misaligned data transfers may span two data cache entries. In this case, the processor checks for a hit one entry at a time. Therefore, it is possible that a portion of the access results in a hit and a portion results in a miss. The hit and miss are treated independently. Figure 6-3 illustrates the organization of the data cache.

The operation of the data cache differs for read and write cycles. A data read cycle operates exactly like an instruction cache read cycle; when a miss occurs, an external cycle is initiated to obtain the operand from memory, and the data is loaded into the cache if the access is cachable. In the case of a misaligned operand that spans two cache entries, two long words are required from memory. Burst mode operation may also be initiated to fill an entire line of the data cache. Read accesses from the CPU address space and address translation table search accesses are not stored in the data cache.

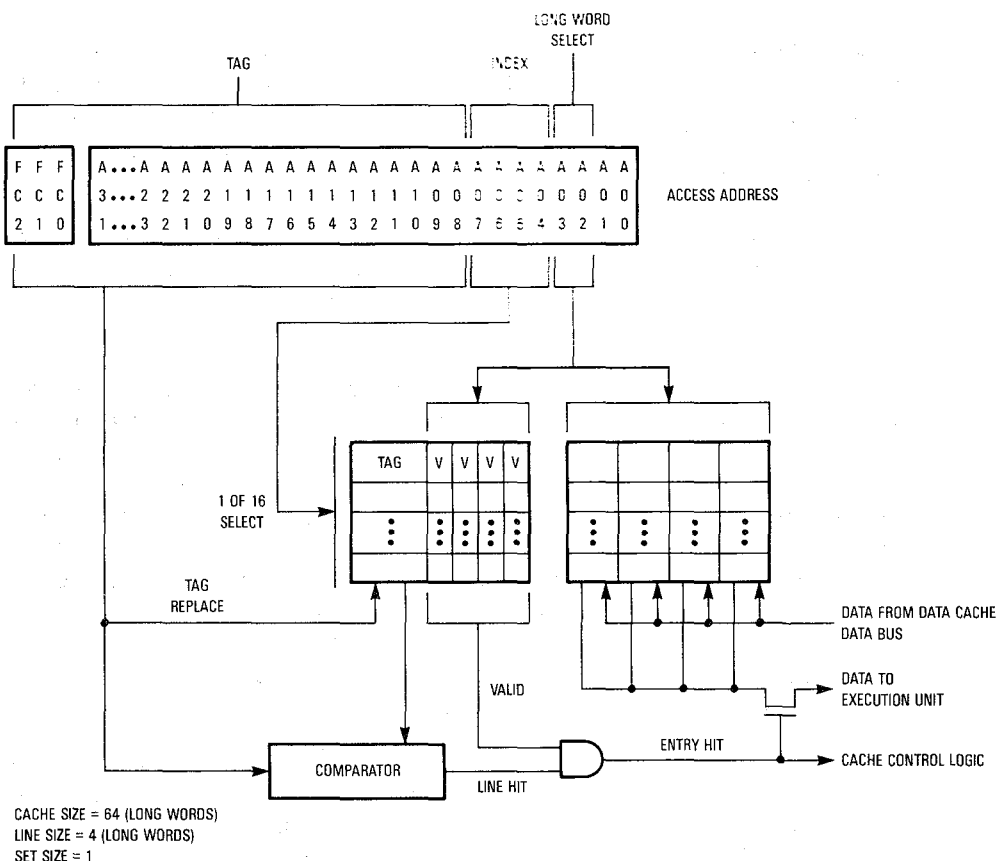


Figure 6-3. On-Chip Data Cache Organization

The data cache on the MC68030 is a writethrough cache. When a hit occurs on a write cycle, the data is written both to the cache and to external memory (provided the MMU validates the access), regardless of the operand size and even if the cache is frozen. If the MMU determines that the access is invalid, the write is aborted, the corresponding entry is invalidated, and a bus error exception is taken. Since the write to the cache completes before the write to external memory, the cache contains the new value even if the external write terminates in a bus error. The value in the data cache might be used by another instruction before the external write cycle has completed, although this should not have any adverse consequences. Refer to **7.6 BUS SYNCHRONIZATION** for the details of bus synchronization.

6.1.2.1 WRITE ALLOCATION. The supervisor program can configure the data cache for either of two types of allocation for data cache entries that miss on write cycles. The state of the write allocation (WA) bit in the cache control register specifies either no write allocation or write allocation with partial validation of the data entries in the cache on writes.

When no write allocation is selected (WA=0), write cycles that miss do not alter the data cache contents. In this mode, the processor does not replace entries in the cache during write operations. The cache is updated only during a write hit.

When write allocation is selected (WA=1), the processor always updates the data cache on cachable write cycles, but only validates an updated entry that hits or an entry that is updated with long-word data that is long-word aligned. When a tag miss occurs on a write of long-word data that is long-word aligned, the corresponding tag is replaced, and only the long word being written is marked as valid. The other three entries in the cache line are invalidated when a tag miss occurs on a misaligned long-word write or on a byte or word write, the data is not written in the cache, the tag is unaltered, and the valid bit(s) are cleared. Thus, an aligned long-word data write may replace a previously valid entry; whereas, a misaligned data write or a write of data that is not long word may invalidate a previously valid entry or entries.

Write allocation eliminates stale data that may reside in the cache because of either of two unique situations: multiple mapping of two or more logical addresses to one physical address within the same task or allowing the same physical location to be accessed by both supervisor and user mode cycles. Stale data conditions can arise when operating in the no-write-allocation mode and all the following conditions are satisfied:

- Multiple mapping (object aliasing) is allowed by the operating system.
- A read cycle loads a value for an “aliased” physical address into the data cache.
- A write cycle occurs, referencing the same aliased physical object as above but using a different logical address, causing a cache miss and no update to the cache (has the same page offset).
- The physical object is then read using the first alias, which provides stale data from the cache.

In this case, the data in the cache no longer matches that in physical memory and is stale. Since the write-allocation mode updates the cache during write cycles, the data in the cache remains consistent with physical memory. Note that when $\overline{\text{CIOUT}}$ is asserted, the data cache is completely ignored, even on write cycles operating in the write-allocation mode. Also note that since the $\overline{\text{CIIN}}$ signal is ignored on write cycles, cache entries may be created for noncacheable data (when $\overline{\text{CIIN}}$ is asserted on a write) when operating in the write-allocation mode. Figure 6-4 shows the manner in which each mode operates in five different situations.

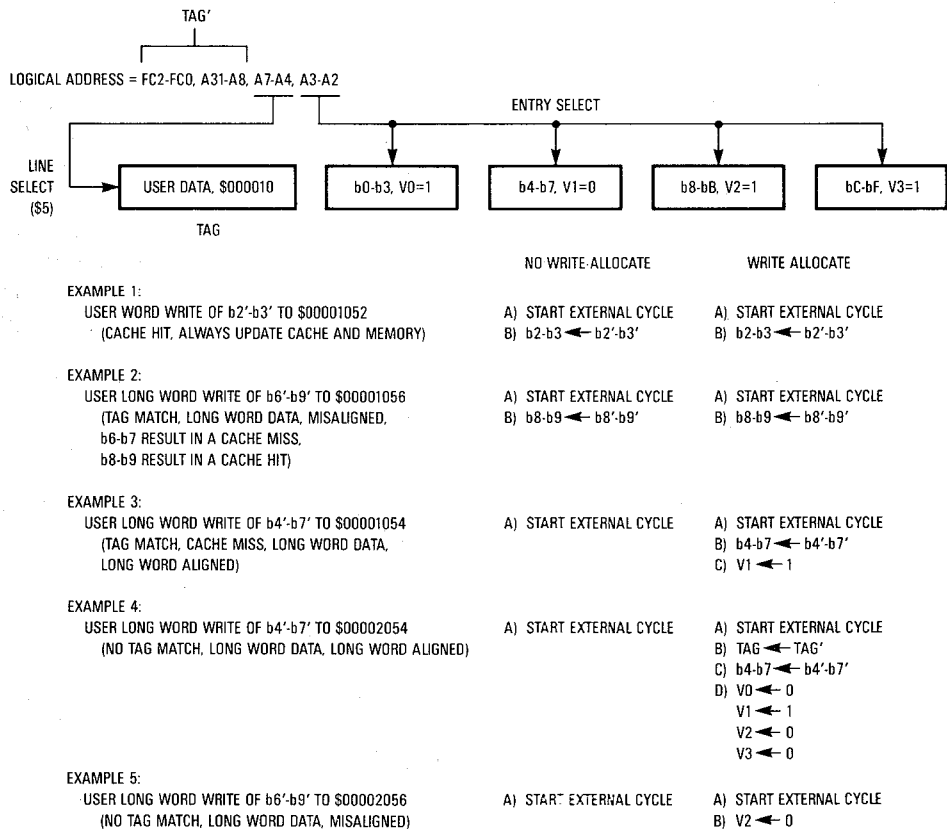


Figure 6-4. No-Write-Allocation and Write-Allocation Mode Examples

6.1.2.2 READ-MODIFY-WRITE ACCESSES. The read portion of a read-modify-write cycle is always forced to miss in the data cache. However, if the system allows internal caching of read-modify-write cycle operands ($\overline{\text{CIOUT}}$ and $\overline{\text{CIIN}}$ both negated), the processor either uses the data read from memory to update a matching entry in the data cache or creates a new entry with the read data in the case of no matching entry. The write portion of a read-modify-write operation also updates a matching entry in the data cache. In the case of a cache miss on the write, the allocation of a new cache entry for the data being written is controlled by the WA bit. Table search accesses, however, are completely ignored by the data cache; it is never updated for a table search access.

6.1.3 Cache Filling

The bus controller can load either cache in either of two ways:

- Single entry mode
- Burst fill mode

In the single entry mode, the bus controller loads a single long-word entry of a cache line. In the burst fill mode, an entire line (four long words) can be filled. Refer to **SECTION 7 BUS OPERATION** for detailed information about the bus cycles required for both modes.

6.1.3.1 SINGLE ENTRY MODE. When a cachable access is initiated and a burst mode operation is not requested by the MC68030 or is not supported by external hardware, the bus controller transfers a single long word for the corresponding cache entry. An entire long word is required. If the port size of the responding device is smaller than 32 bits, the MC68030 executes all bus cycles necessary to fill the long word.

When a device cannot supply its entire port width of data, regardless of the size of the transfer, the responding device must consistently assert the cache inhibit input ($\overline{\text{CIIN}}$) signal. For example, a 32-bit port must always supply 32 bits, even for 8- and 16-bit transfers; a 16-bit port must supply 16 bits, even for 8-bit transfers. The MC68030 assumes that a 32-bit termination signal for the bus cycle indicates availability of 32 valid data bits, even if only 16 or 8 bits are requested. Similarly, the processor assumes that a 16-bit termination signal indicates that all 16 bits are valid. If the device cannot supply its full port width of data, it must assert $\overline{\text{CIIN}}$ for all bus cycles corresponding to a cache entry.

When a cachable read cycle provides data with both $\overline{CIIN}$ and $\overline{BERR}$ negated, the MC68030 attempts to fill the cache entry. Figure 6-5 shows the organization of a line of data in the caches. The notation b0, b1, b2, and so forth identifies the bytes within the line. For each entry in the line, a valid bit in the associated tag corresponds to a long-word entry to be loaded. Since a single valid bit applies to an entire long word, a single entry mode operation must provide a full 32 bits of data. Ports less than 32 bits wide require several read cycles for each entry.

Figure 6-5 shows an example of a byte data operand read cycle starting at byte address \$03 from an 8-bit port. Provided the data item is cachable, this operation results in four bus cycles. The first cycle requested by the MC68030 reads a byte from address \$03. The 8-bit $\overline{DSACKx}$ response causes the MC68030 to fetch the remainder of the long word starting at address \$00. The bytes are latched in the following order: b3, b0, b1, and b2. Note that during cache loading operations, devices must indicate the same port size consistently throughout all cycles for that long-word entry in the cache.

Figure 6-6 shows the access of a byte data operand from a 16-bit port. This operation requires two read cycles. The first cycle requests the byte at address \$03. If the device responds with a 16-bit $\overline{DSACKx}$ encoding, the word at address \$02 (including the requested byte) is accepted by the MC68030. The second cycle requests the word at address \$00. Since the device again responds with a 16-bit $\overline{DSACKx}$ encoding, the remaining two bytes of the long word are latched, and the cache entry is filled.

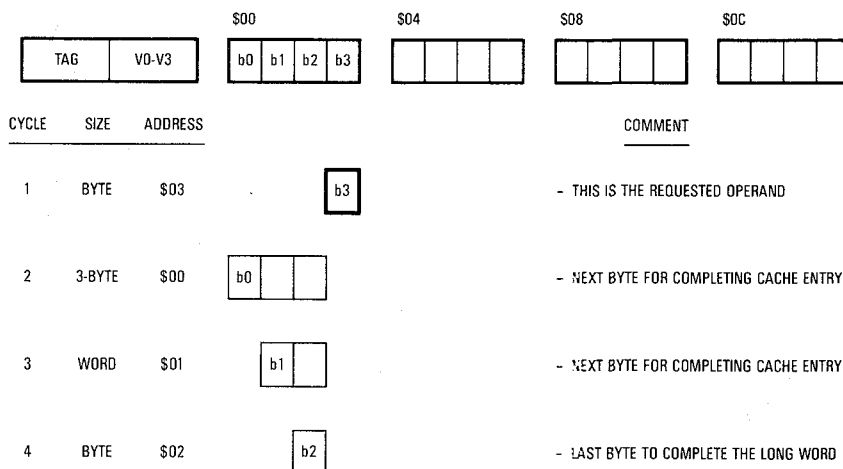


Figure 6-5. Single Entry Mode Operation — 8-Bit Port

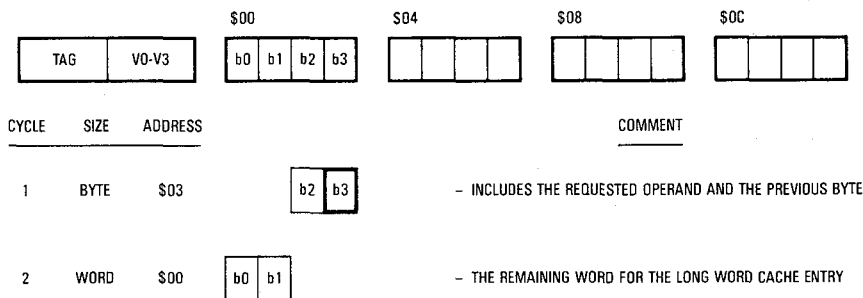


Figure 6-6. Single Entry Mode Operation — 16-Bit Port

With a 32-bit port, the same operation is shown in Figure 6-7. Only one read cycle is required. All four bytes (including the requested byte) are latched during the cycle.

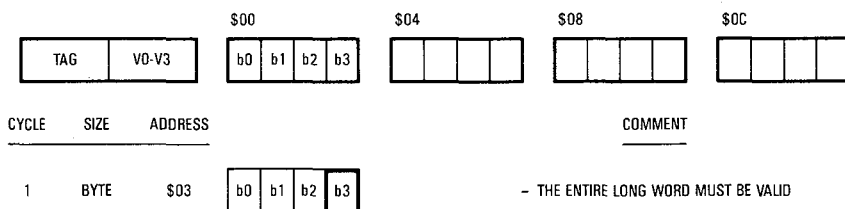


Figure 6-7. Single Entry Mode Operation — 32-Bit Port

If a requested access is misaligned and spans two cache entries, the bus controller attempts to fill both associated long-word cache entries. An example of this is an operand request for a long word on an odd-word boundary. The MC68030 first fetches the initial byte(s) of the operand (residing in the first long word) and then requests the remaining bytes to fill that cache entry (if the port size is less than 32 bits) before it requests the remainder of the operand and corresponding long word to fill the second cache entry. If the port size is 32 bits, the processor performs two accesses, one for each cache entry.

Figure 6-8 shows a misaligned access of a long word at address \$06 from an 8-bit port requiring eight bus cycles to complete. Reading this long-word operand requires eight read cycles, since accesses to all eight addresses return 8-bit port-size encodings. These cycles fetch the two cache entries that the requested long-word spans. The first cycle requests a long word at address \$06 and accepts the first requested byte (b6). The subsequent transfers of the first long word are performed in the following order: b7, b4, b5. The remaining four read cycles transfer the four bytes of the second cache entry. The sequence of access for the entire operation is b6, b7, b4, b5, b8, b9, bA, and bB.

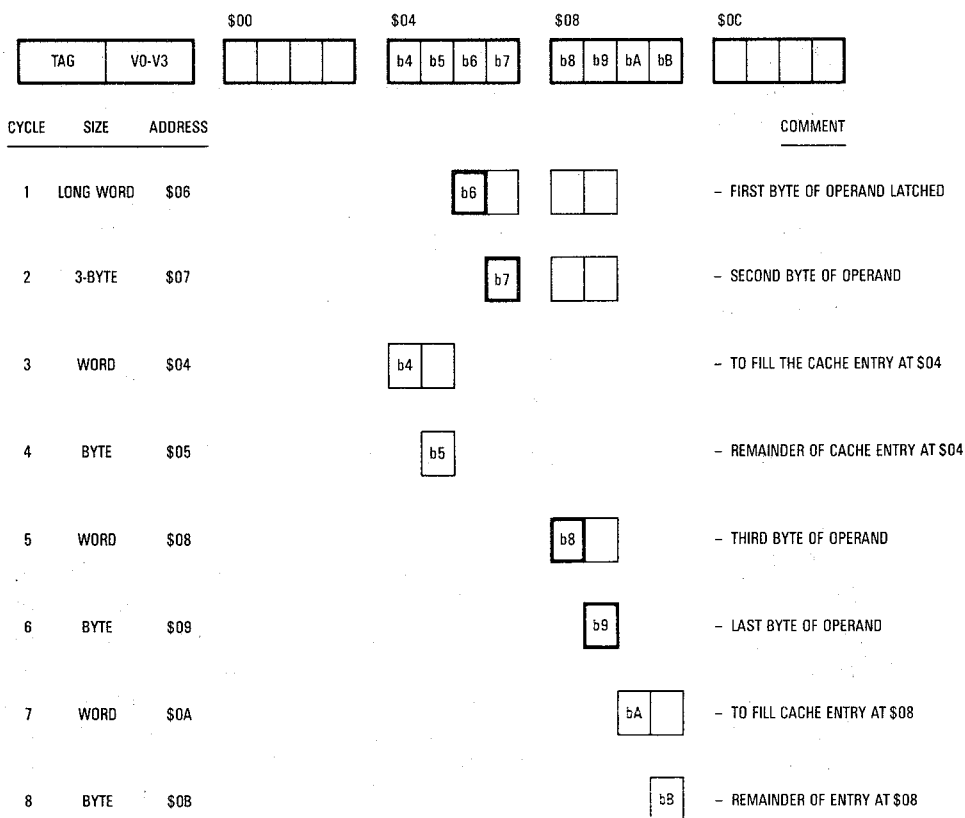


Figure 6-8. Single Entry Mode Operation — Misaligned Long Word and 8-Bit Port

The next example, shown in Figure 6-9, is a read of a misaligned long-word operand from devices that return 16-bit $\overline{DSACKx}$ encodings. The processor accepts the first portion of the operand, the word from address \$06, and requests a word from address \$04 to fill the cache entry. Next, the processor reads the word at address \$08, the second portion of the operand, and stores it in the cache also. Finally, the processor accesses the word at \$0A to fill the second long-word cache entry.

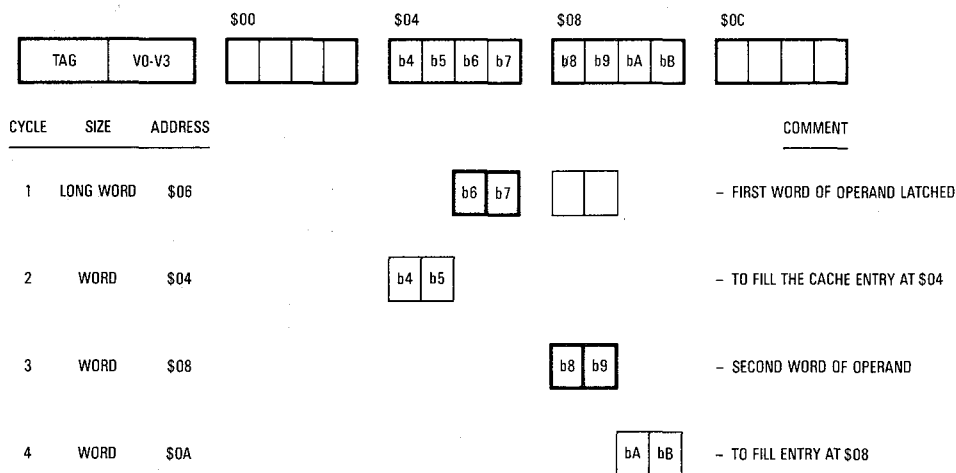


Figure 6-9. Single Entry Mode Operation — Misaligned Long Word and 16-Bit Port

Two read cycles are required for a misaligned long-word operand transfer from devices that return 32-bit $\overline{DSACKx}$ encodings. As shown in Figure 6-10, the first read cycle requests the long word at address \$06 and latches the long word at address \$04. The second read cycle requests and latches the long word corresponding to the second cache entry at address \$08. Two read cycles are also required if $\overline{STERM}$ is used to indicate a 32-bit port instead of the 32-bit $\overline{DSACKx}$ encoding.

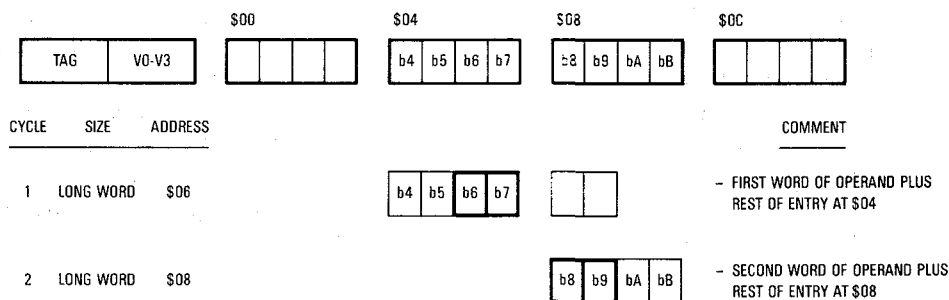


Figure 6-10. Single Entry Mode Operation — Misaligned Long Word and 32-Bit DSACKx Port

If all bytes of a long word are cachable, $\overline{CIIN}$ must be negated for all bus cycles required to fill the entry. If any byte is not cachable, $\overline{CIIN}$ must be asserted for all corresponding bus cycles. The assertion of the $\overline{CIIN}$ signal prevents the caches from being updated during read cycles. Write cycles (including the write portion of a read-modify-write cycle) ignore the assertion of the $\overline{CIIN}$ signal and may cause the data cache to be altered, depending on the state of the cache (whether or not the write cycle hits), the state of the WA bit in the CACR, and the conditions indicated by the MMU.

The occurrence of a bus error while attempting to load a cache entry aborts the entry fill operation but does not necessarily cause a bus error exception. If the bus error occurs on a read cycle for a portion of the required operand (not the remaining bytes of the cache entry) to be loaded into the data cache, the processor immediately takes a bus error exception. If the read cycle in error is made only to fill the data cache (the data is not part of the target operand), no exception occurs, but the corresponding entry is marked invalid. For the instruction cache, the processor marks the entry as invalid, but only takes an exception if the execution unit attempts to use the instruction word(s).

6.1.3.2 BURST MODE FILLING. Burst mode filling is enabled by bits in the cache control register. The data burst enable bit must be set to enable burst filling of the data cache. Similarly, the instruction burst enable bit must be set to enable burst filling of the instruction cache. When burst filling is enabled and the corresponding cache is enabled, the bus controller requests a burst mode fill operation in either of these cases:

- A read cycle for either the instruction or data cache misses due to the indexed tag not matching.
- A read cycle tag matches, but all long words in the line are invalid.

he bus controller requests a burst mode fill operation by asserting the cache burst request signal ($\overline{\text{CBREQ}}$). The responding device may sequentially supply one to four long words of cachable data, or it may assert the cache inhibit input signal ($\overline{\text{CIIN}}$) when the data in a long word is not cachable. If the responding device does not support the burst mode and it terminates cycles with $\overline{\text{STERM}}$, it should not acknowledge the request with the assertion of the cache burst acknowledge ($\overline{\text{CBACK}}$) signal. The MC68030 ignores the assertion of $\overline{\text{CBACK}}$ during cycles terminated with $\overline{\text{DSACKx}}$.

The cache burst request signal ($\overline{\text{CBREQ}}$) requests burst mode operation from the referenced external device. To operate in the burst mode, the device or external hardware must be able to increment the low-order address bits if required, and the current cycle must be a 32-bit synchronous transfer ($\overline{\text{STERM}}$ must be asserted) as described in **SECTION 7 BUS OPERATION**. The device must also assert $\overline{\text{CBACK}}$ (at the same time as $\overline{\text{STERM}}$) at the end of the cycle in which the MC68030 asserts $\overline{\text{CBREQ}}$. $\overline{\text{CBACK}}$ causes the processor to continue driving the address and bus control signals and to latch a new data value for the next cache entry at the completion of each subsequent cycle (as defined by $\overline{\text{STERM}}$), for a total of up to four cycles (until four long words have been read).

When a cache burst is initiated, the first cycle attempts to load the cache entry corresponding to the instruction word or data item explicitly requested by the execution unit. The subsequent cycles are for the subsequent entries in the cache line. In the case of a misaligned transfer when the operand spans two cache entries within a cache line, the first cycle corresponds to the cache entry containing the portion of the operand at the lower address.

Figure 6-11 illustrates the four cycles of a burst operation and shows that the second, third, and fourth cycles are run in burst mode. A distinction is made between the first cycle of a burst operation and the subsequent cycles because the first cycle is requested by the microsequencer and the burst fill cycles are requested by the bus controller. Therefore, when data from the first cycle is returned, it is immediately available for the execution unit (EU). However, data from the burst fill cycles is not available to the EU until the burst operation is complete. Since the microsequencer makes two separate requests for misaligned data operands, only the first portion of the misaligned operand returned during a burst operation is available to the EU after the first cycle is complete. The microsequencer must wait for the burst operation to complete before requesting the second portion of the operand. Normally, the request for the second portion results in a data cache hit unless the second cycle of the burst operation terminates abnormally.

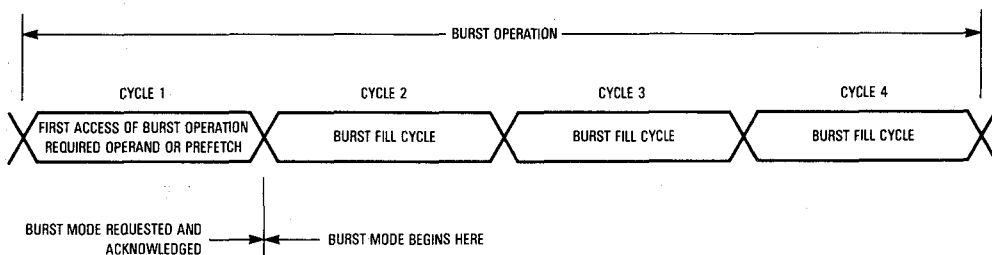


Figure 6-11. Burst Operation Cycles and Burst Mode

The bursting mechanism allows addresses to wrap around so that the entire four long words in the cache line can be filled in a single burst operation, regardless of the initial address and operand alignment. Depending on the structure of the external memory system, address bits A2 and A3 may have to be incremented externally to select the long words in the proper order for loading into the cache. The MC68030 holds the entire address bus constant for the duration of the burst cycle. Figure 6-12 shows an example of this address wraparound. The initial cycle is a long-word access from address \$6. Because the responding device returns CBACK and STERM (signaling a 32-bit port), the entire long word at base address \$04 is transferred. Since the initial address is \$06 when CBREQ is asserted, the next entry to be burst filled into the cache should correspond to address \$08, then \$0C, and last, \$00. This addressing is compatible with existing nibble-mode dynamic RAMs, and can be supported by page and static column modes with an external modulo 4 counter for A2 and A3.

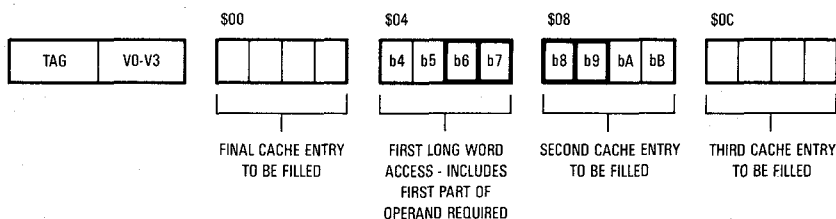


Figure 6-12. Burst Filling Wraparound Example

The MC68030 does not assert $\overline{\text{CBREQ}}$ during the first portion of a misaligned access if the remainder of the access does not correspond to the same cache line. Figure 6-13 shows an example in which the first portion of a misaligned access is at address \$0F. With a 32-bit port, the first access corresponds to the cache entry at address \$0C, which is filled using a single-entry load operation. The second access, at address \$10 corresponding to the second cache line, requests a burst fill and the processor asserts $\overline{\text{CBREQ}}$. During this burst operation, long words \$10, \$14, \$18, and \$1C are all filled in that order.

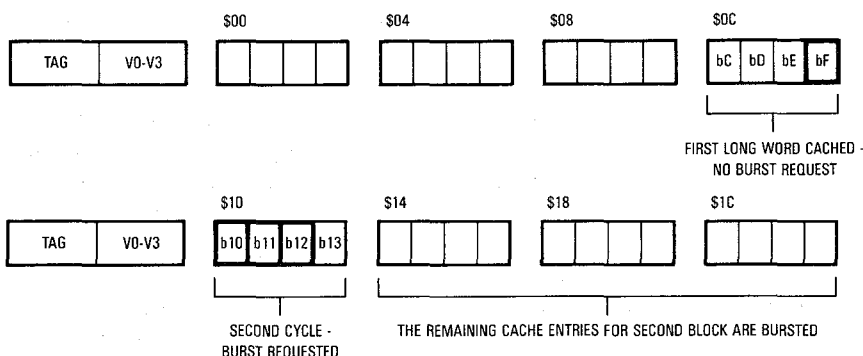


Figure 6-13. Deferred Burst Filling Example

The processor does not assert $\overline{\text{CBREQ}}$ if any of the following conditions exist:

- The appropriate cache is not enabled
- Burst filling for the cache is not enabled
- The cache freeze bit for the appropriate cache is set
- The current operation is the read portion of a read-modify-write operation
- The MMU has inhibited caching for the current page
- The cycle is for the first access of an operand that spans two cache lines (crosses a modulo 16 boundary)

Additionally, the assertion of $\overline{\text{CIIN}}$ and $\overline{\text{BERR}}$ and the premature negation of $\overline{\text{CBACK}}$ affect burst operation as described in the following paragraphs.

The assertion of $\overline{\text{CIIN}}$ during the first cycle of a burst operation causes the data to be latched by the processor, and if the requested operand is aligned (the entire operand is latched in the first cycle), the data is passed on to the instruction pipe or execution unit. However, the data is not loaded into its corresponding cache. In addition, the MC68030 negates $\overline{\text{CBREQ}}$, and the burst operation is aborted. If a portion of the requested operand remains to be read (due to misalignment), a second read cycle is initiated at the appropriate address with $\overline{\text{CBREQ}}$ negated.

The assertion of $\overline{\text{CIIN}}$ during the second, third, or fourth cycle of a burst operation prevents the data during that cycle from being loaded into the appropriate cache and causes $\overline{\text{CBREQ}}$ to negate, aborting the burst operation. However, if the data for the cycle contains part of the requested operand, the execution unit uses that data.

The premature negation of the $\overline{\text{CBACK}}$ signal during the burst operation causes the current cycle to complete normally, loading the data successfully transferred into the appropriate cache. However, the burst operation aborts and $\overline{\text{CBREQ}}$ negates.

A bus error occurring during a burst operation also causes the burst operation to abort. If the bus error occurs during the first cycle of a burst (i.e., before burst mode is entered), the data read from the bus is ignored, and the entire associated cache line is marked "invalid". If the access is a data cycle, exception processing proceeds immediately. If the cycle is for an instruction fetch, a bus error exception is made pending. This bus error is processed only if the execution unit attempts to use either instruction word. Refer to **11.2.2 Instruction Pipe** for more information about pipeline operation.

For either cache, when a bus error occurs after the burst mode has been entered (that is, on the second cycle or later), the cache entry corresponding to that cycle is marked invalid, but the processor does not take an exception (the microsequencer has not yet requested the data). In the case of an instruction cache burst, the data from the aborted cycle is completely ignored. Pending instruction prefetches are still pending and are subsequently run by the processor. If the second cycle is for a portion of a misaligned data operand fetch and a bus error occurs, the processor terminates the burst operation and negates $\overline{\text{CBREQ}}$. Once the burst terminates, the microsequencer requests a read cycle for the second portion. Since the burst terminated abnormally for the second cycle of the burst, the data cache results in a miss, and a second external cycle is required. If $\overline{\text{BERR}}$ is again asserted, the MC68030 then takes an exception.

On the initial access of a burst operation, a “retry” (indicated by the assertion of $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$) causes the processor to retry the bus cycle and assert $\overline{\text{CBREQ}}$ again. However, signaling a retry with simultaneous $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ during the second, third, or fourth cycle of a burst operation does not cause a retry operation, even if the requested operand is misaligned. Assertion of $\overline{\text{BERR}}$ and $\overline{\text{HALT}}$ during burst fill cycles of a burst operation causes independent bus error and halt operations. The processor remains halted until $\overline{\text{HALT}}$ is negated, and then handles the bus error as described in the previous paragraphs.

6.2 CACHE RESET

When a hardware reset of the processor occurs, all valid bits of both caches are cleared. The cache enable bits, burst enable bits, and the freeze bits in the cache control register (CACR) for both caches (refer to Figure 6-14) are also cleared, effectively disabling both caches. The WA bit in the CACR is also cleared.

6.3 CACHE CONTROL

Only the MC68030 cache control circuitry can directly access the cache arrays, but the supervisor program can set bits in the CACR to exercise control over cache operations. The supervisor also has access to the cache address register (CAAR), which contains the address for a cache entry to be cleared.

6.3.1 Cache Control Register

The CACR, shown in Figure 6-14, is a 32-bit register that can be written or read by the MOVEC instruction or indirectly modified by a reset. Five of the bits (4–0) control the instruction cache; six other bits (13–8) control the data cache. Each cache is controlled independently of the other, although a similar operation can be performed for both caches by a single MOVEC instruction. For example, loading a long word in which bits 3 and 11 are set into the CACR clears both caches. Bits 31–14 and 7–5 are reserved for Motorola definition. They are currently read as zeros and are ignored when written. For future compatibility, writes should not set these bits.

31	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		WA	DBE	CD	CED	FD	ED	0	0	0	IBE	CI	CEI	FI	EI

WA = Write Allocate
 DBE = Data Burst Enable
 CD = Clear Data Cache
 CED = Clear Entry in Data Cache
 FD = Freeze Data Cache
 ED = Enable Data Cache
 IBE = Instruction Burst Enable
 CI = Clear Instruction Cache
 CEI = Clear Entry in Instruction Cache
 FI = Freeze Instruction Cache
 EI = Enable Instruction Cache

Figure 6-14. Cache Control Register

6.3.1.1 WRITE ALLOCATE. Bit 13, the WA bit, is set to select the write-allocation mode (refer to **6.1.2.1 WRITE ALLOCATION**) for write cycles. Clearing this bit selects the no-write-allocation mode. A reset operation clears this bit. The supervisor should set this bit when it shares data with the user task or when any task maps multiple logical addresses to one physical address. If the data cache is disabled or frozen, the WA bit is ignored.

6.3.1.2 DATA BURST ENABLE. Bit 12, the DBE bit, is set to enable burst filling of the data cache. Operating systems and other software set this bit when burst filling of the data cache is desired. A reset operation clears the DBE bit.

6.3.1.3 CLEAR DATA CACHE. Bit 11, the CD bit, is set to clear all entries in the data cache. Operating systems and other software set this bit to clear data from the cache prior to a context switch. The processor clears all valid bits in the data cache at the time a MOVEC instruction loads a one into the CD bit of the CACR. The CD bit is always read as a zero.

6.3.1.4 CLEAR ENTRY IN DATA CACHE. Bit 10, the CED bit, is set to clear an entry in the data cache. The index field of the CAAR (see Figure 6-15) corresponding to the index and long-word select portion of an address specifies the entry to be cleared. The processor clears only the specified long word by clearing the valid bit for the entry at the time a MOVEC instruction loads a one into the CED bit of the CACR, regardless of the states of the ED and FD bits. The CED bit is always read as a zero.

6.3.1.5 FREEZE DATA CACHE. Bit 9, the FD bit, is set to freeze the data cache. When the FD bit is set and a miss occurs during a read or write of the data cache, the indexed entry is not replaced. However, write cycles that hit in the data cache cause the entry to be updated even when the cache is frozen. When the FD bit is clear, a miss in the data cache during a read cycle causes the entry (or line) to be filled, and the filling of entries on writes that miss are then controlled by the WA bit. A reset operation clears the FD bit.

6.3.1.6 ENABLE DATA CACHE. Bit 8, the ED bit, is set to enable the data cache. When it is cleared, the data cache is disabled. A reset operation clears the ED bit. The supervisor normally enables the data cache, but it can clear ED for system debugging or emulation, as required. Disabling the data cache does not flush the entries. If it is enabled again, the previously valid entries remain valid and can be used.

6.3.1.7 INSTRUCTION BURST ENABLE. Bit 4, the IBE bit, is set to enable burst filling of the instruction cache. Operating systems and other software set this bit when burst filling of the instruction cache is desired. A reset operation clears the IBE bit.

6.3.1.8 CLEAR INSTRUCTION CACHE. Bit 3, the CI bit, is set to clear all entries in the instruction cache. Operating systems and other software set this bit to clear instructions from the cache prior to a context switch. The processor clears all valid bits in the instruction cache at the time a MOVEC instruction loads a one into the CI bit of the CACR. The CI bit is always read as a zero.

6.3.1.9 CLEAR ENTRY IN INSTRUCTION CACHE. Bit 2, the CEI bit, is set to clear an entry in the instruction cache. The index field of the CAAR (see Figure 6-15) corresponding to the index and long-word select portion of an address specifies the entry to be cleared. The processor clears only the specified long word by clearing the valid bit for the entry at the time a MOVEC instruction loads a one into the CEI bit of the CACR, regardless of the states of the EI and FI bits. The CEI bit is always read as a zero.

6.3.1.10 FREEZE INSTRUCTION CACHE. Bit 1, the FI bit, is set to freeze the instruction cache. When the FI bit is set and a miss occurs in the instruction cache, the entry (or line) is not replaced. When the FI bit is cleared to zero, a miss in the instruction cache causes the entry (or line) to be filled. A reset operation clears the FI bit.

6.3.1.11 ENABLE INSTRUCTION CACHE. Bit 0, the EI bit, is set to enable the instruction cache. When it is cleared, the instruction cache is disabled. A reset operation clears the EI bit. The supervisor normally enables the instruction cache, but it can clear EI for system debugging or emulation, as required. Disabling the instruction cache does not flush the entries. If it is enabled again, the previously valid entries remain valid and may be used.

6.3.2 Cache Address Register

The CAAR is a 32-bit register shown in Figure 6-15. The index field (bits 7–2) contains the address for the “clear cache entry” operations. The bits of this field correspond to bits 7–2 of addresses; they specify the index and a long word of a cache line. Although only the index field is used currently, all 32 bits of the register are implemented and are reserved for use by Motorola.

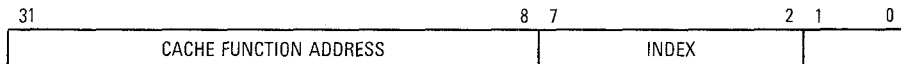


Figure 6-15. Cache Address Register

